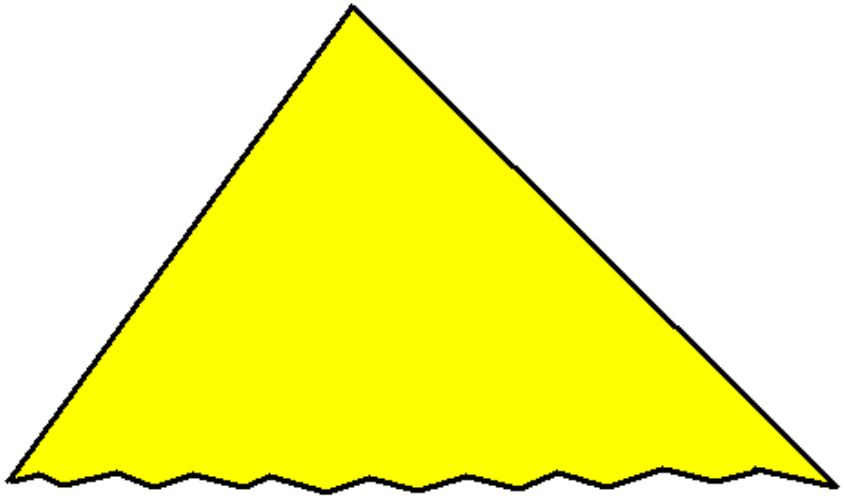
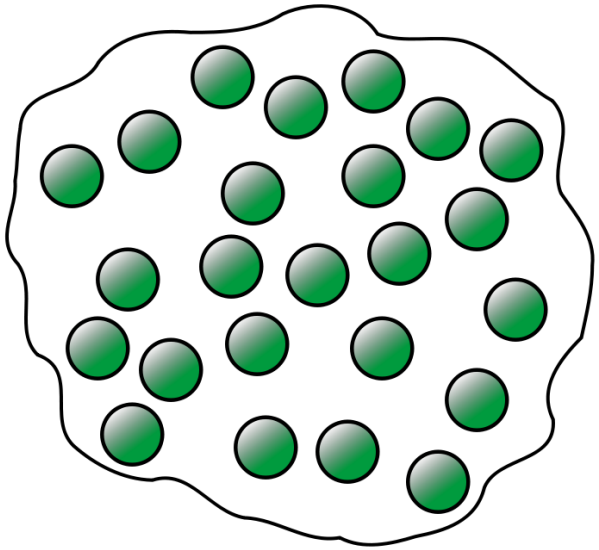
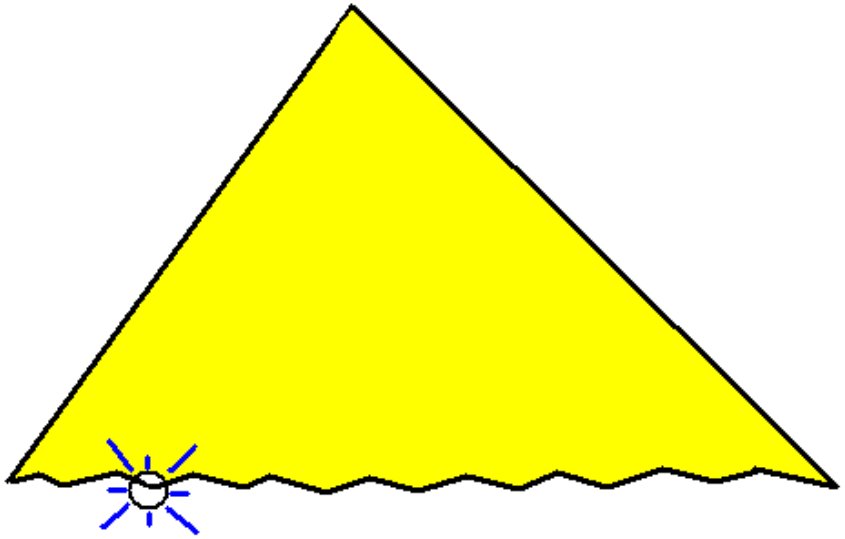
# Local Search

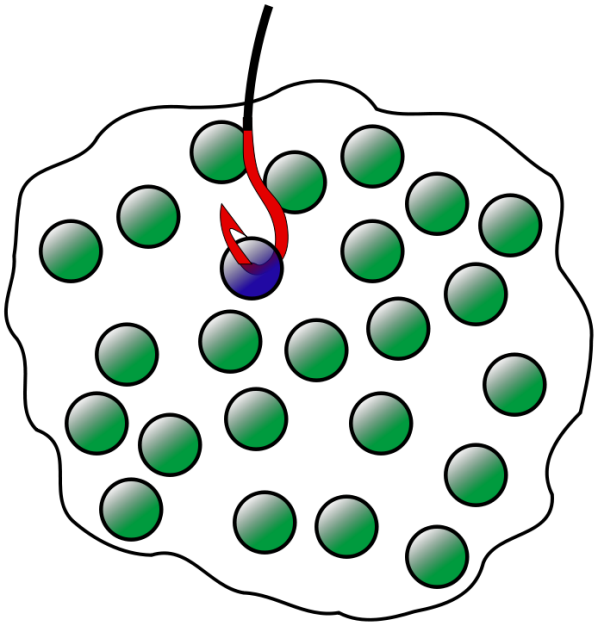
Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . . ).
All of them are based on random choices.



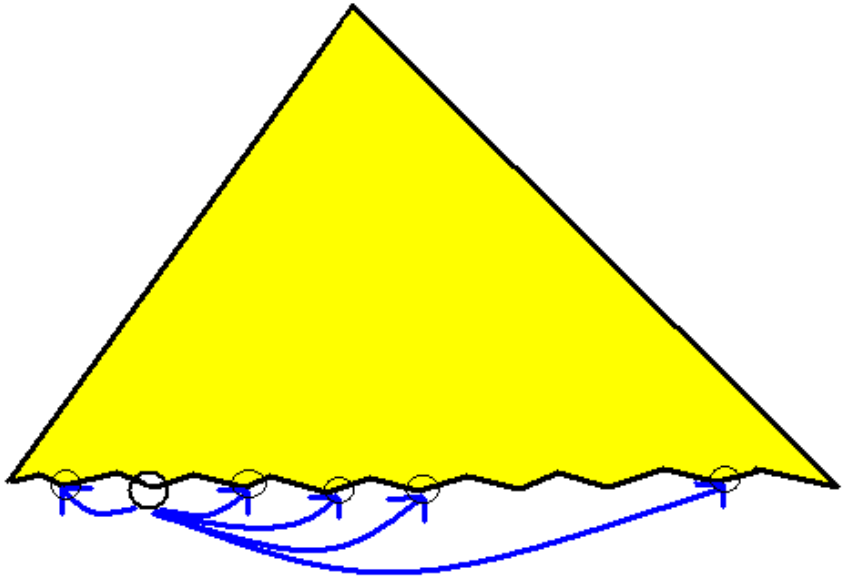


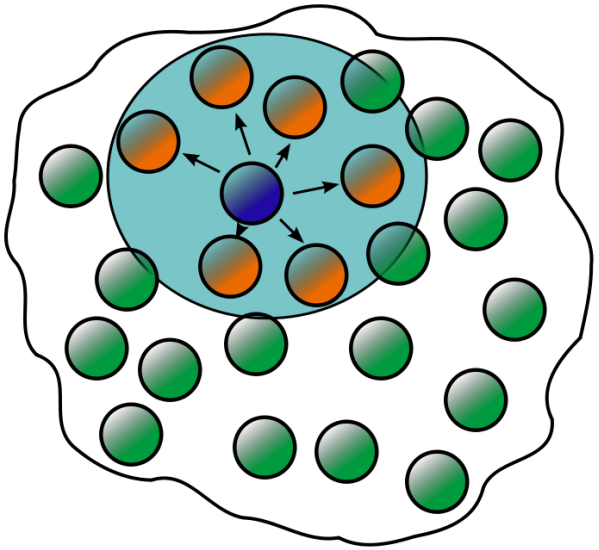
Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . . ).
All of them are based on random choices.



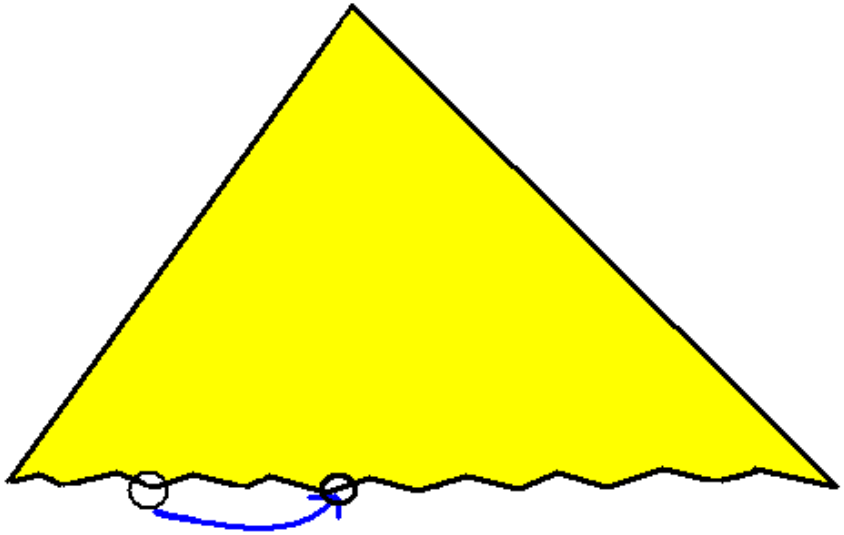


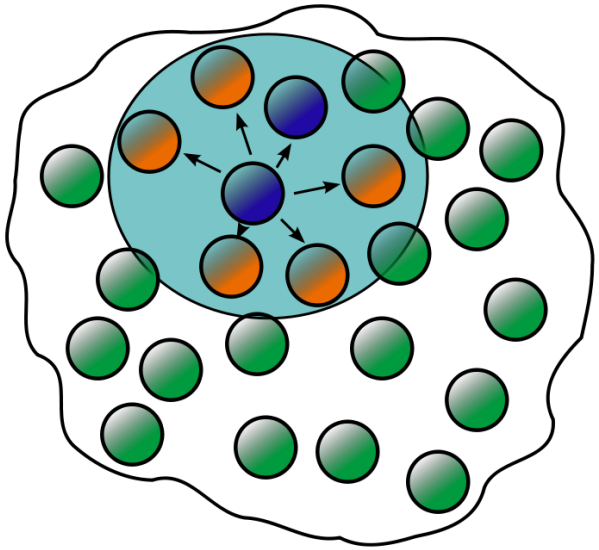
Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . . ).
All of them are based on random choices.



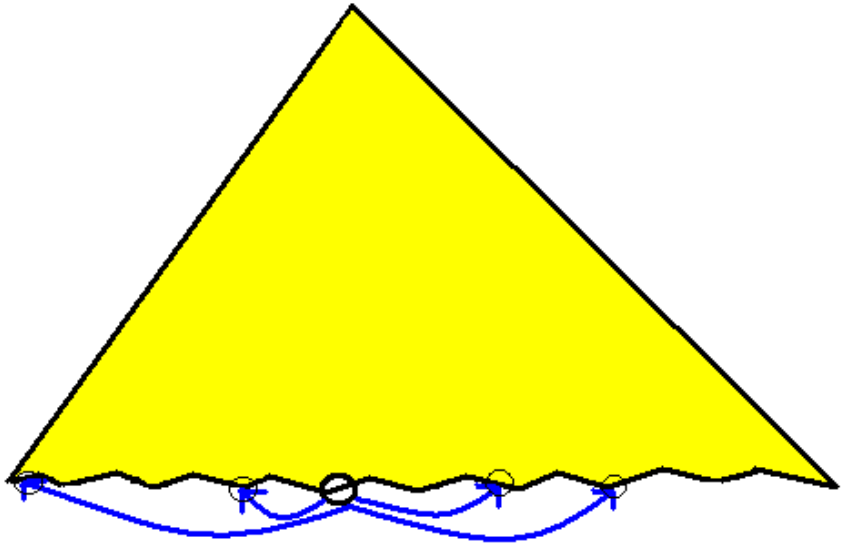


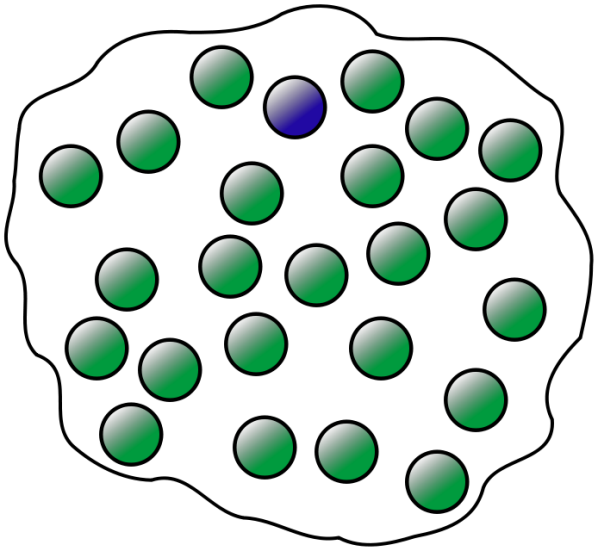
Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . . ).
All of them are based on random choices.



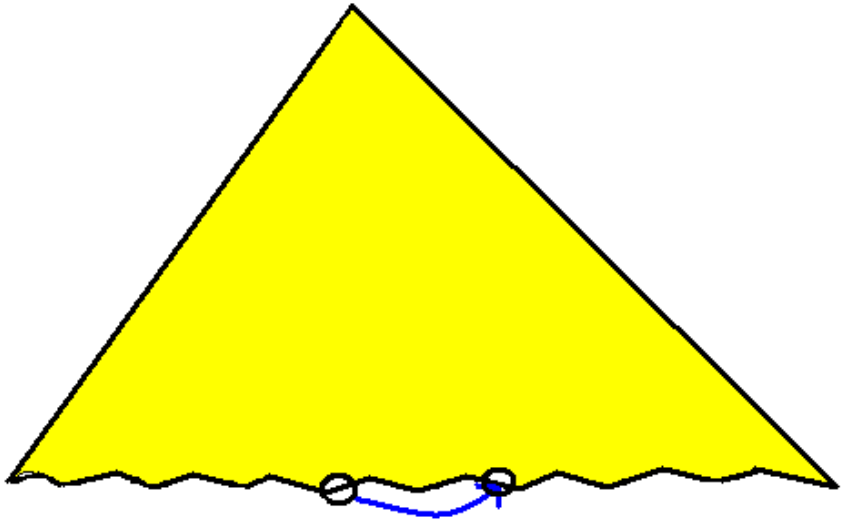


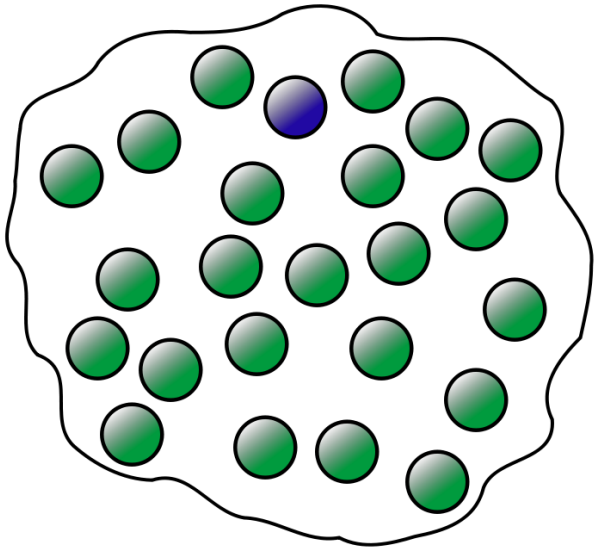
Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . . ).
All of them are based on random choices.



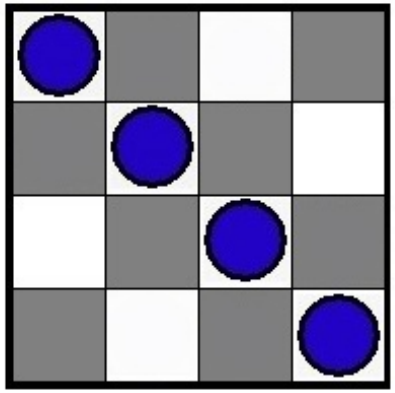


Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . . ).
All of them are based on random choices.



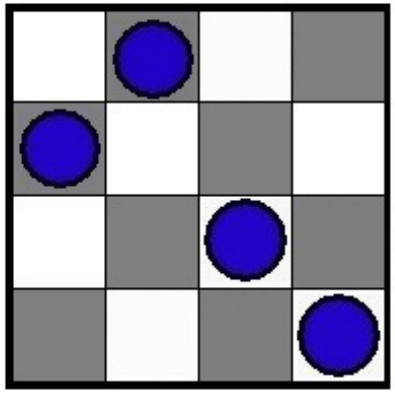


- LS methods can be applied to CSP.
- Remove some constraints and count a penalty anytime those constraints are violated. We find more admissible solutions!
- Less constraints $\rightarrow$ easier to find an initial solution
- Then run the LS algorithm. If the optimum has value 0 then all the constraints are satisfied.
- Let us see an example with the Queens. Let us remove the diagonal constraints and define as $f$ the number of attacks (i.e. number of violated diagonal constraints).

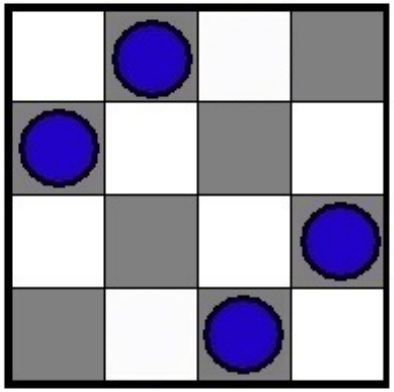


12 attacks

$\Rightarrow$ swap(1,2)

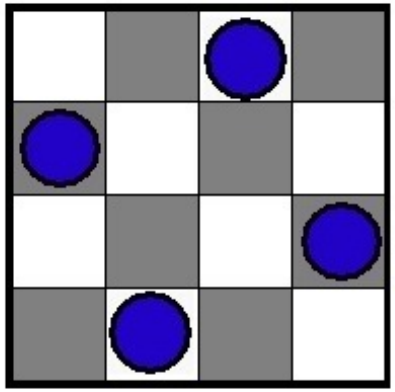


4 attacks $\Downarrow$ swap(3,4)

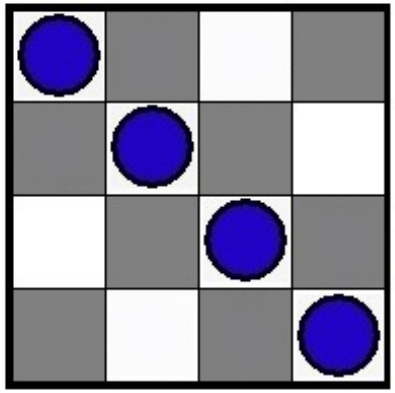


8 attacks

$\Leftarrow$ swap(2,3)

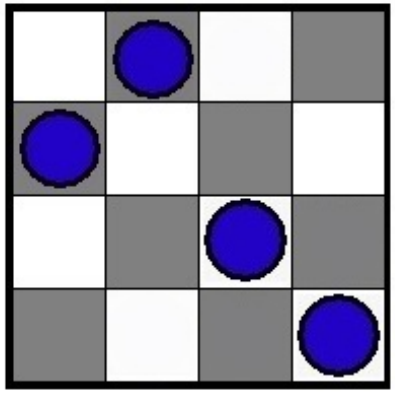


0 attacks

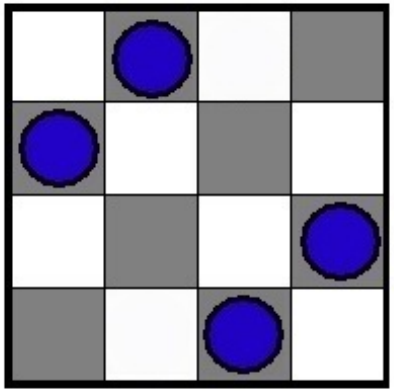


12 attacks

$\Rightarrow$ swap(1,2)

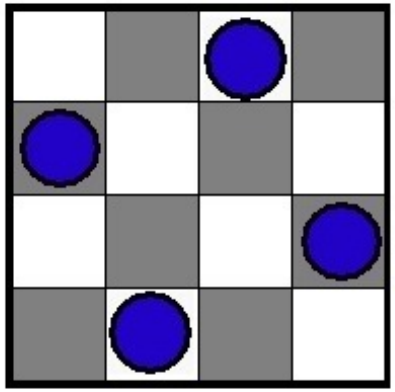


4 attacks $\Downarrow$ swap(3,4)

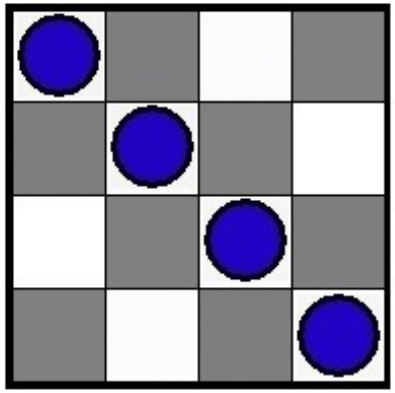


8 attacks

$\Leftarrow$ swap(2,3)

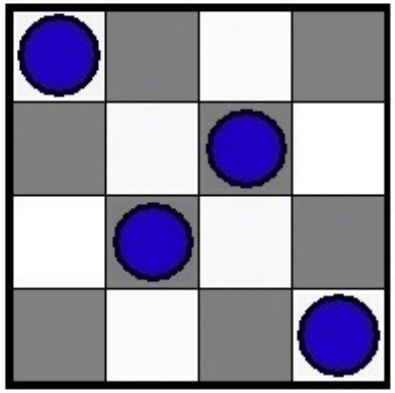


0 attacks

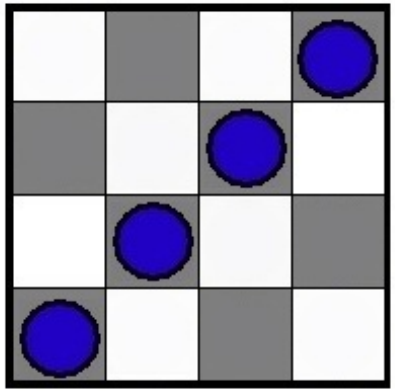


12 attacks

$\Rightarrow$ swap(2,3)

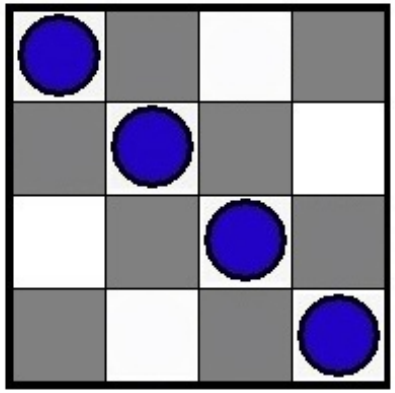


4 attacks $\Downarrow$ swap(1,4)



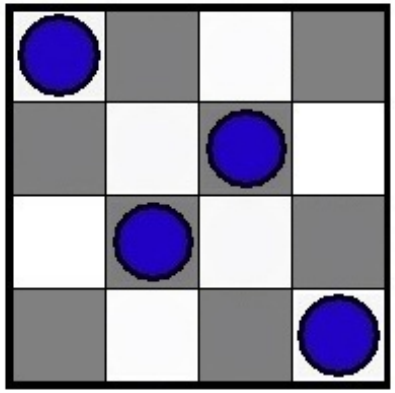
12 attacks

Not every choice improves the violations (+ local minima!)

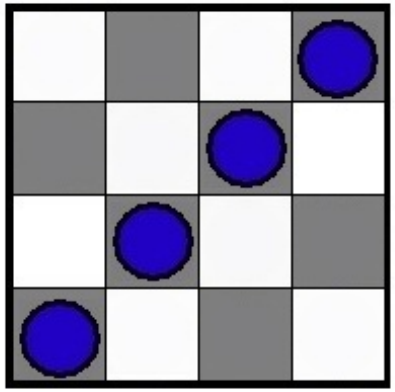


12 attacks

$\Rightarrow$ swap(2,3)



4 attacks $\Downarrow$ swap(1,4)



12 attacks

Not every choice improves the violations (+ local minima!)

Define:

- An algorithm for finding the initial solution
- A function $N$ that computes the Neighborhood of a solution $\sigma$ that identifies a set $N(\sigma)$ of feasible solutions similar to $\sigma$
- If possible, the Neighborhood is characterized by a simple *move* (e.g., change/swap of values between two variables)
- Moreover:
  - How to select a move
  - How to explore the Neighborhood
  - How to avoid undesirable solutions/situations (e.g., loops)
  - How/When halting the computation (typically, after a fixed number of non-improving steps)

- Given a solution $\sigma$, compute $N(\sigma)$.
  - Hill Climbing: choose $\sigma' \in N(\sigma)$ minimizing locally $f$, namely $f(\sigma') = \min\{f(\mu) \,:\, \mu \in N(\sigma)\}$
  - Simulated Annealing:
    - Let $T$ (temperature) be a gloabal variable slowly decreasing during computation.
    - $*$ Chose randomly $\sigma' \in N(\sigma)$.
    - If $f(\sigma') < f(\sigma)$ take $\sigma'$ as new solution.
    - Otherwise, generate a random number $p \in [0, 1]$.
    - Let $F$ be a function in $[0, 1]$ that increases as $T$ decreases and that also computes the distance between $f(\sigma)$ and $f(\sigma')$.
    - If $p > F(T, f(\sigma), f(\sigma'))$ then take $\sigma'$ as new solution, otherwise go to $*$ (choose a new random $\sigma' \in N(\sigma)$ and repeat last items)

- Monte Carlo: It is a sort of Simulated Annealing with fixed temperature and $F = 0.5$.
  *The name Monte Carlo was coined in the 1940s by Stanislaw Ulam, Enrico Fermi, John von Neumann, and Nicholas Metropolis among others working on nuclear weapon projects in the Los Alamos Labs.*
- Tabu Search: It is a Simulated Annealing, with the difference that you store a list (tabu list) if the last $k$ solutions in order to avoid loops.
- Others ...

- Monte Carlo: It is a sort of Simulated Annealing with fixed temperature and $F = 0.5$.
  *The name Monte Carlo was coined in the 1940s by Stanislaw Ulam, Enrico Fermi, John von Neumann, and Nicholas Metropolis among others working on nuclear weapon projects in the Los Alamos Labs.*
- Tabu Search: It is a Simulated Annealing, with the difference that you store a list (tabu list) if the last $k$ solutions in order to avoid loops.
- Others ...

# Local Search

## When?

- Discrete optimization problems where the notion of swap is natural (e.g., timetables, scheduling, etc.)
- Continuous problems (e.g., simulations)
- Easy to implement. However, better using an ad hoc tool (e.g. EasyLocal) and focusing on the problem and on the meta-heuristics.

# (Integer) Linear Programming

## Integer Linear Programming

minimize $c_1X_1 + c_2X_2 + \cdots + c_nX_n$
subject to

$$a_{1,1}X_1 + a_{1,2}X_2 + \cdots + a_{1,n}X_n \; op \; b_1$$
$$a_{2,1}X_1 + a_{2,2}X_2 + \cdots + a_{2,n}X_n \; op \; b_2$$
$$\cdots$$
$$a_{m,1}X_1 + a_{m,2}X_2 + \cdots + a_{m,n}X_n \; op \; b_m$$
$$X_1, \ldots, X_n \in \mathbb{N}$$

*op* is typically $\leq$.
$=$ and $\geq$ are easy to support, while for $<$ and $>$ some work has to be made.
It is a COP with constraints and a linear cost function($f$).

## Integer Linear Programming

minimize $c_1X_1 + c_2X_2 + \cdots + c_nX_n$
subject to

$$a_{1,1}X_1 + a_{1,2}X_2 + \cdots + a_{1,n}X_n \; op \; b_1$$
$$a_{2,1}X_1 + a_{2,2}X_2 + \cdots + a_{2,n}X_n \; op \; b_2$$
$$\cdots$$
$$a_{m,1}X_1 + a_{m,2}X_2 + \cdots + a_{m,n}X_n \; op \; b_m$$
$$X_1, \ldots, X_n \in \mathbb{N}$$

*op* is typically $\leq$.
$=$ and $\geq$ are easy to support, while for $<$ and $>$ some work has to be made.
It is a COP with constraints and a linear cost function($f$).

# Linear Programming

## When it is suitable

- A COP in Linear Programming admits a polynomial solution (Elipssoid, Karmarkar, Simplex).
- If your COP is in this class use absolutely LP tools (e.g., CPLEX, DASH, MINTO, OSL, Qsopt, GLPK, . . . )

# Integer Linear Programming

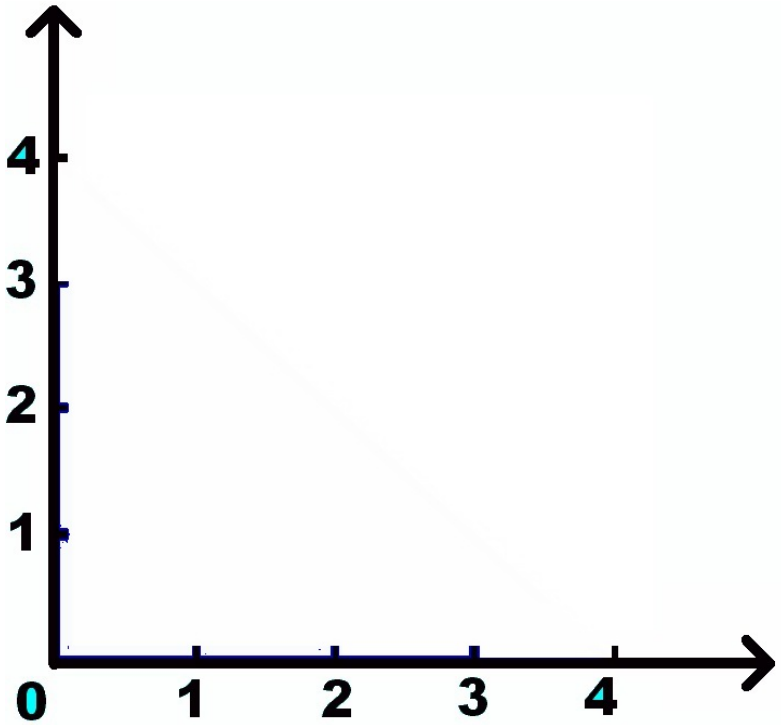
## When it may be suitable

- An ILP COP represents in general a NP complete problem
- The above tools of the previous slide call iteratively a LP problem without the integrality constraints (relaxation). In general this can take exponential time.
- The encoding is an art. Lots of intermediate techniques are used for a good modeling.
- If specifications of the problem are refined over time, better use other techniques (eg, CP!)

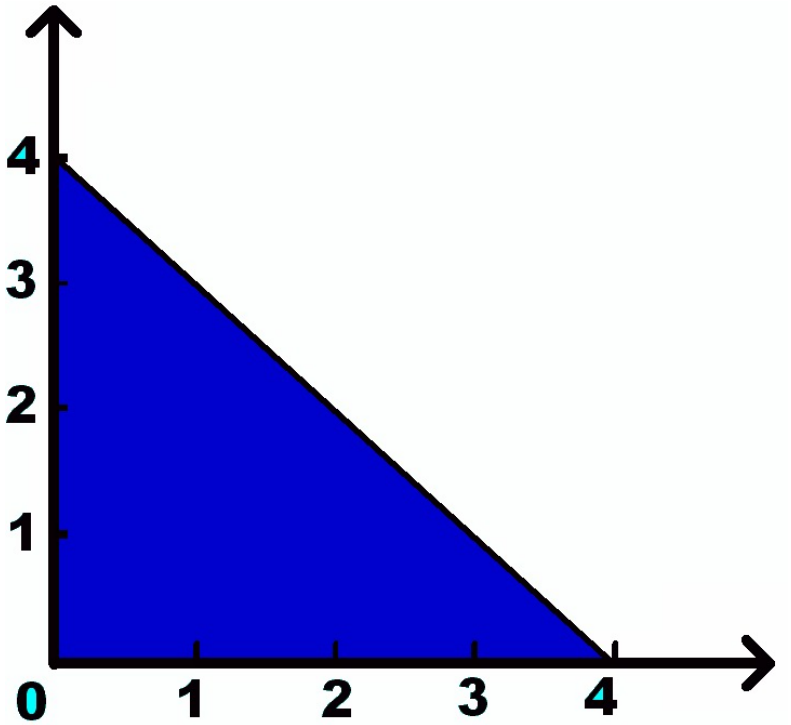
- For solving the COP:
  minimize $C\vec{X}$, subject to $P \equiv A\vec{X} \leq B$, where $\vec{X} \in \mathbb{N}$
- A recursive function can be defined (at the beginning $K = +\infty$)

```
function IP(P, C, K)
    let (z, x⃗, flag) = LP(P,C)
%% z minimum, x⃗ solution, flag states if there is a solution.
    if not(flag) or z ≥ K return K
    elseif integer(x⃗) return z
    else pick a non-integer variable X_i from x
        K = IP(P ∧ X_i ≤ ⌊x_i⌋, C, K)
        K = IP(P ∧ X_i ≥ ⌈x_i⌉, C, K)
        return K
```

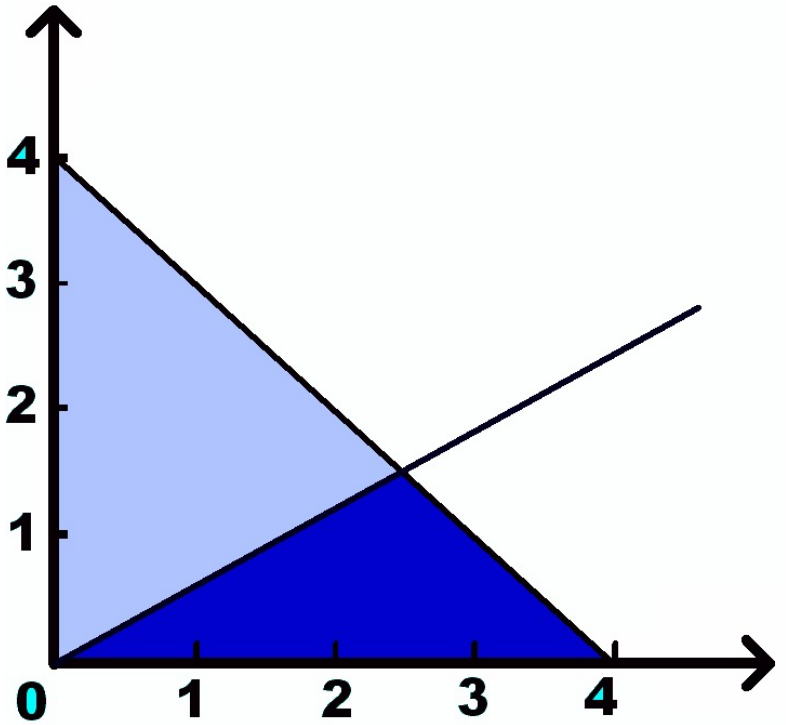
minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.



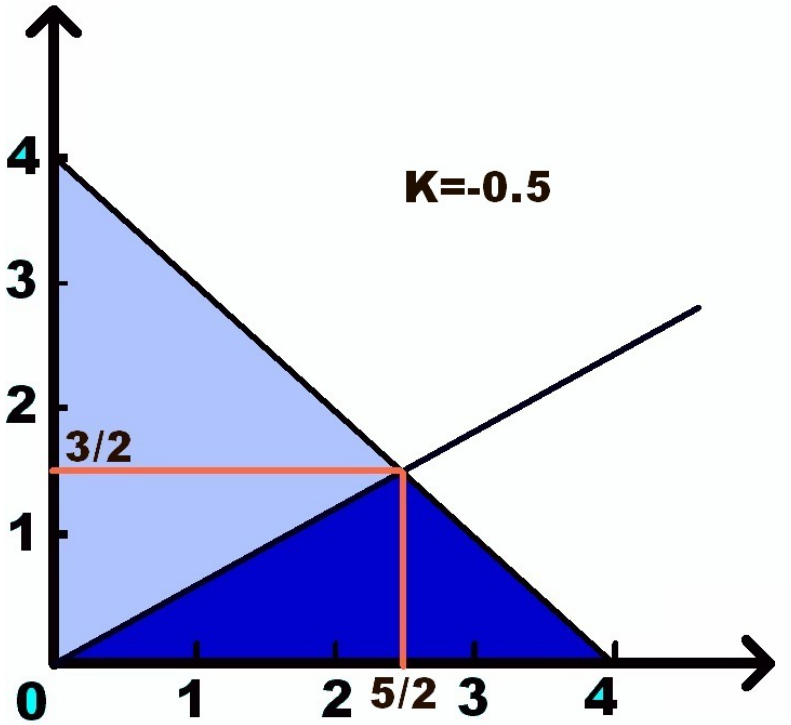
minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.



minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.

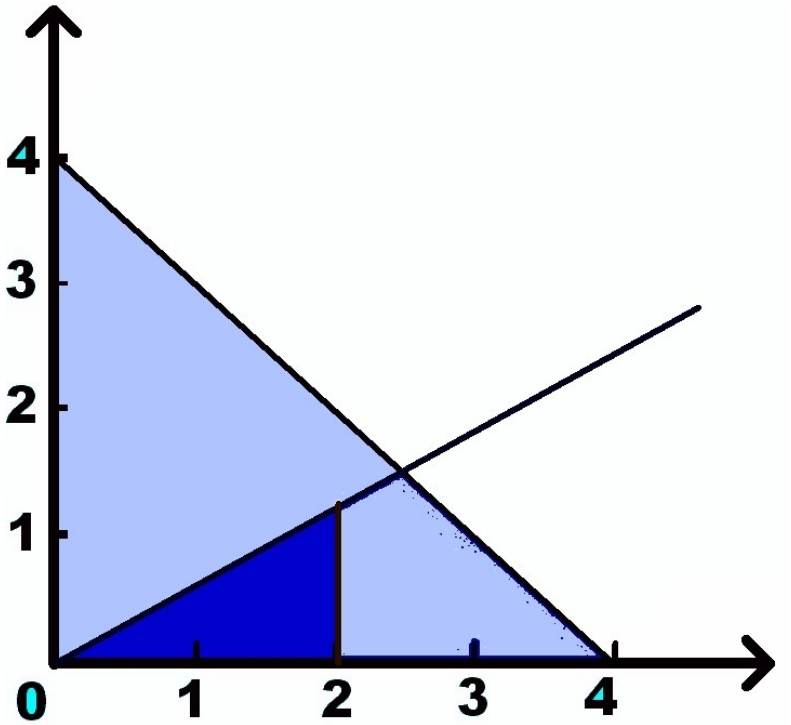


minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.



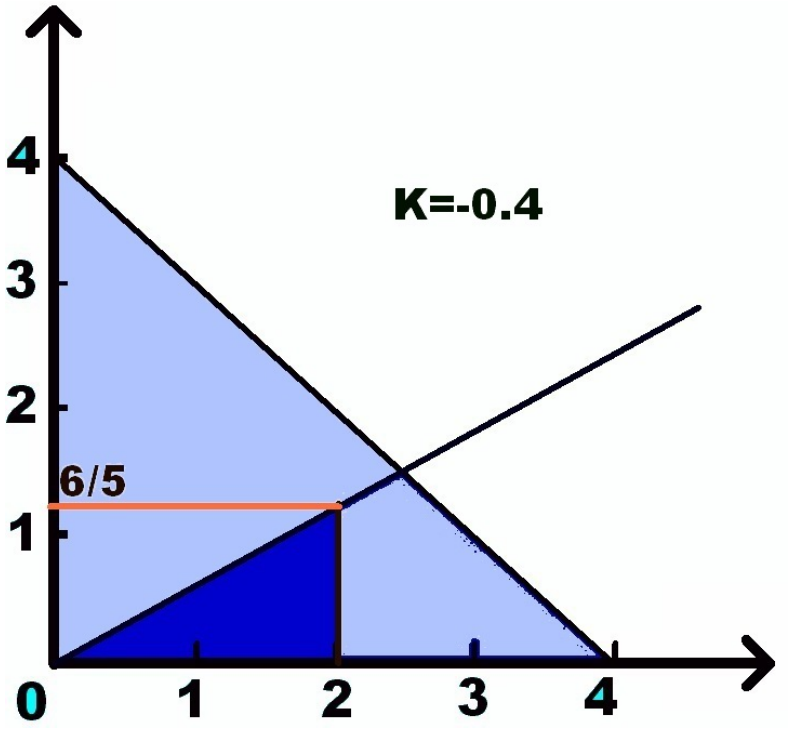

LP

minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.



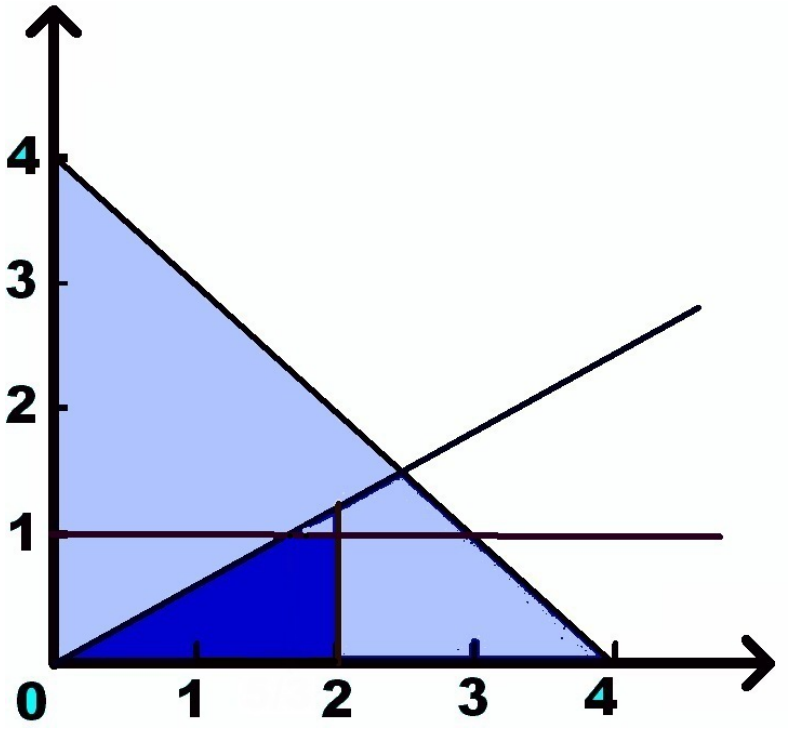

$X \leq 2$

minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.



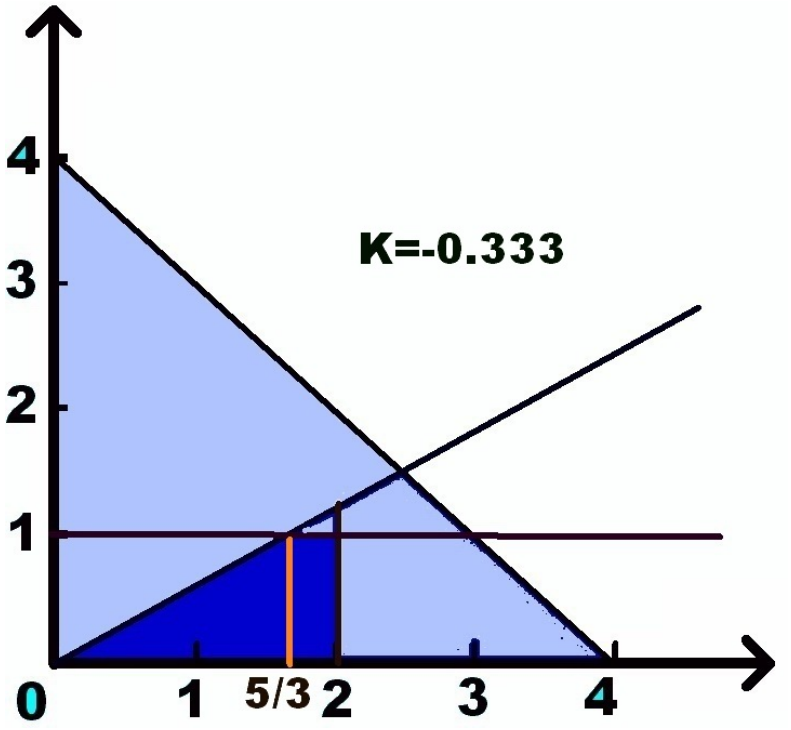

LP

minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.



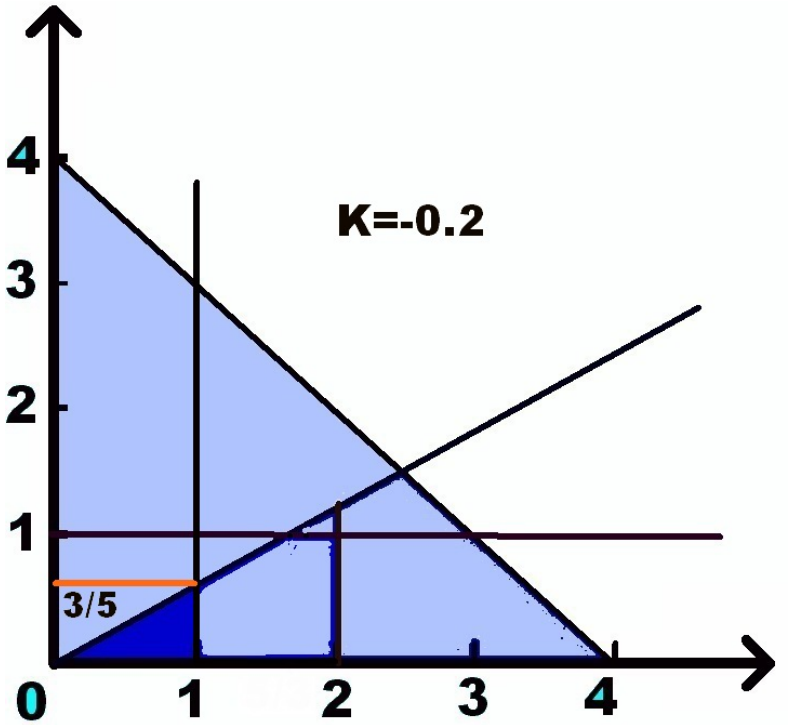

$X \leq 2, Y \leq 1$

minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.



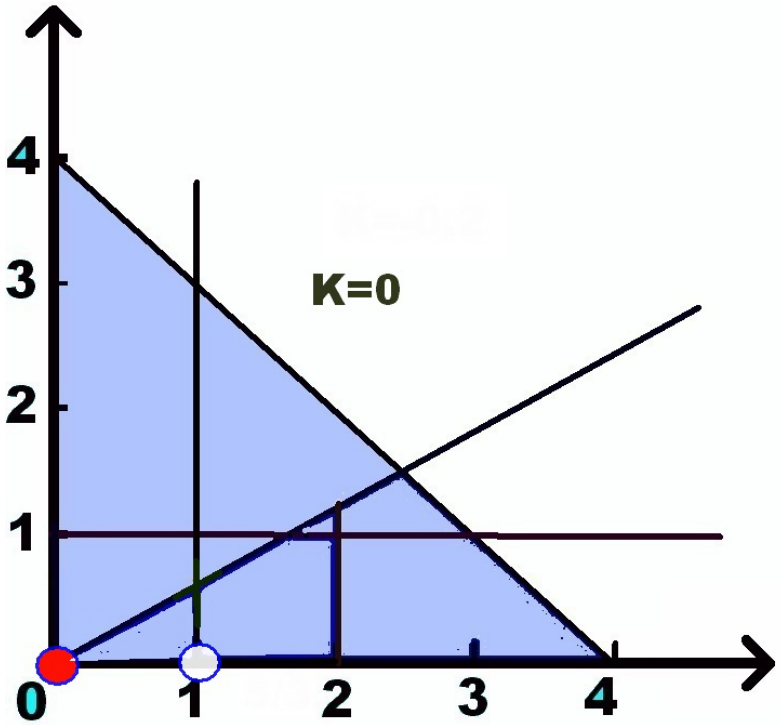

LP

minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.



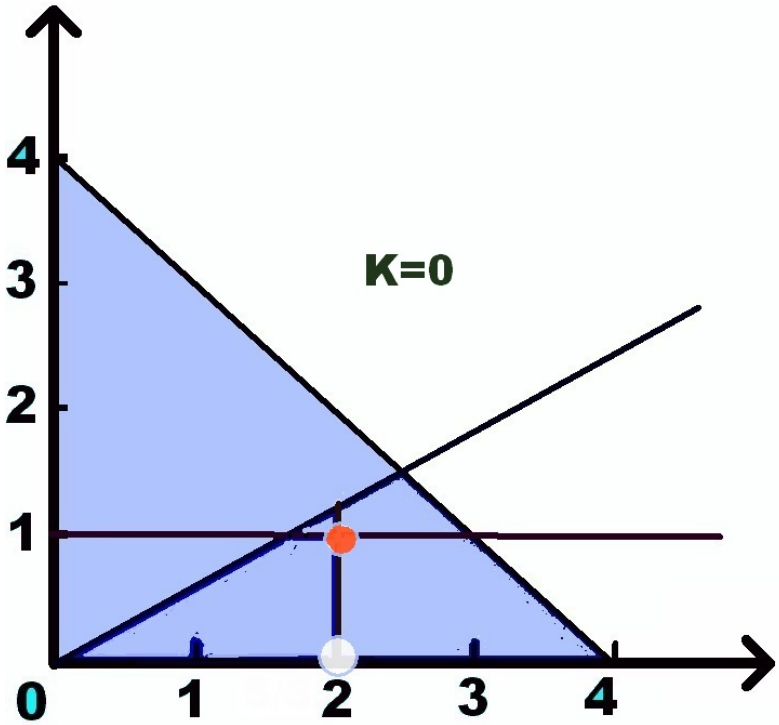

$X \leq 1, Y \leq 1$ –LP

minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.



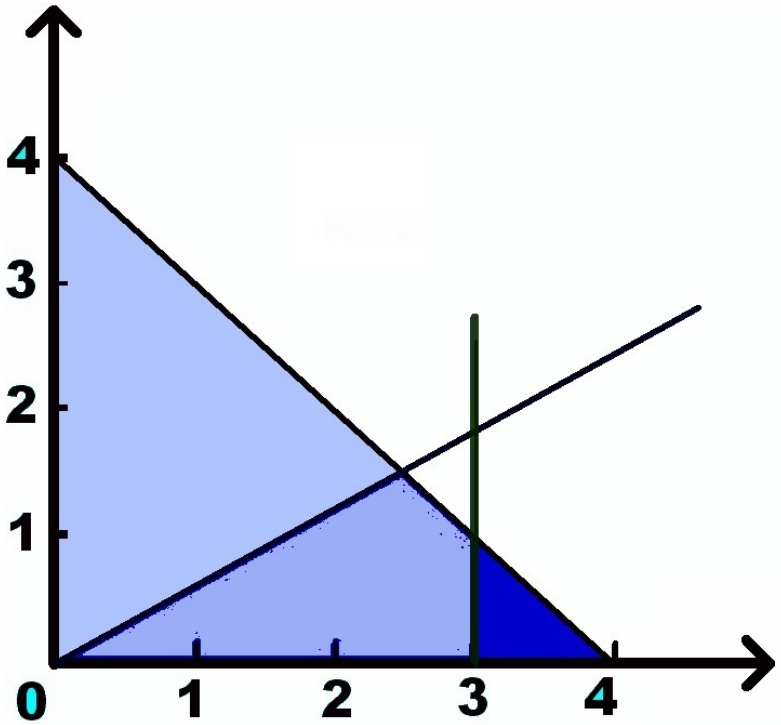

$X \leq 1, Y \leq 0$ –LP: feasible solution

minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.



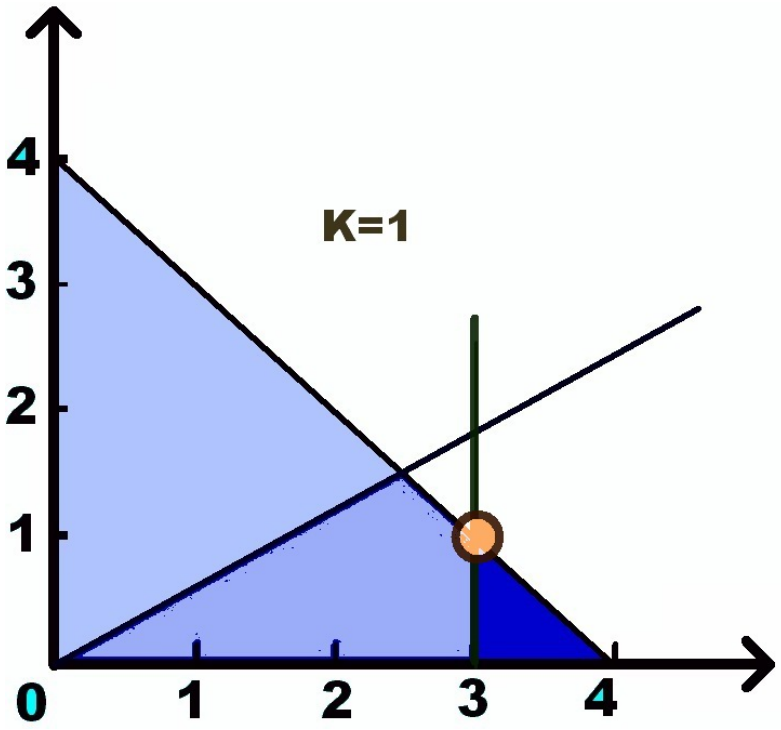

$X \geq 2, X \leq 2, Y \leq 1$ — LP: feasible solution

minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.




$X \geq 3$

minimize $X - 2Y$ subject to
$X \geq 0, Y \geq 0, X + Y \leq 4, 3X - 5Y \geq 0, X, Y \in \mathbb{N}$.




LP: CUT
(never mind if $(X, Y)$ are integer or not)

# Constraint Solving

# Constraint Satisfaction Problem

## Complete Definition

- A CSP is a triple $\langle \mathcal{V}, \mathcal{D}, C \rangle$ where
- $\mathcal{V} = \{V_1, \ldots, V_n\}$ is a set of variables, with domains $\mathcal{D} = \{\mathcal{D}_1, \ldots, \mathcal{D}_n\}$
- $C$ is a set of constraints
- A constraint $c \in C$ on the variables $V_{i_1}, \ldots, V_{i_m}$ is a relation on $\mathcal{D}_{i_1} \times \cdots \times \mathcal{D}_{i_m}$, namely $c \subseteq \mathcal{D}_{i_1} \times \cdots \times \mathcal{D}_{i_m}$
- A solution is an assignment $\sigma : \mathcal{V} \longrightarrow \mathcal{D}_1 \cup \cdots \cup \mathcal{D}_n$ such that
  - for $i = 1, \ldots, n$, $\sigma(V_i) \in \mathcal{D}_i$ and
  - for (each) constraint $c$ in $C$ on $V_{i_1}, \ldots, V_{i_m}$, it holds that $\langle \sigma(V_{i_1}), \ldots, \sigma(V_{i_m}) \rangle \in c$

This definition is syntax-independent.
For COPs, a function $f$ to be minimized.

# CSP/COP

## Example

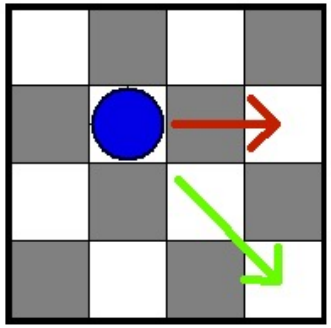
- $\mathcal{V} = \{X_1, X_2, X_3, X_4\}$
- $\mathcal{D}_1 = \mathcal{D}_2 = \mathcal{D}_3 = \mathcal{D}_4 = \{1, 2, 3, 4\}$
- Horizontal constraints: $X_i \neq X_j$ stands for a subset of pairs $(V_i, V_j)$:

$$\{(1,2),(1,3),(1,4), \quad (2,1),(2,3),(2,4), \\ (3,1),(3,2),(3,4), \quad (4,1),(4,2),(4,3)\}$$

- Diagonal constraints: $|X_j - X_i| \neq j - i$.
  For instance, if $i = 2, j = 4$ then:

$$\{(1,1),(1,2),(1,4), \quad (2,1),(2,2),(2,3),$$

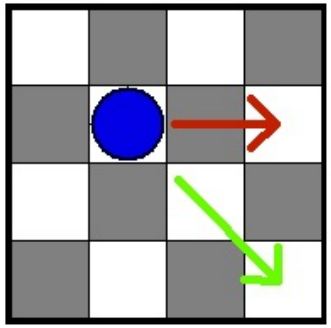
$$(3,2),(3,3),(3,4), \quad (4,1),(4,3),(4,4)\}$$



- If we intersect constraints
- Horizontal constraints: $X_i \neq X_j$ stands for a subset of pairs $(V_i, V_j)$:
  $\{(1,2),(1,3),(1,4), \quad (2,1),(2,3),(2,4),$
  $(3,1),(3,2),(3,4), \quad (4,1),(4,2),(4,3)\}$
- Diagonal constraints: $|X_j - X_i| \neq j - i$.
  For instance, if $i = 2, j = 4$ then:

$$\{(1,1),(1,2),(1,4), \quad \underline{(2,1),(2,2),(2,3),}$$

$$(3,2),(3,3),(3,4), \quad (4,1),(4,3),(4,4)\}$$



$(2,2)$ and $(2,4)$ are not admitted

## CONSTRAINT SOLVER

A constraint solver is a procedure capable of transforming a CSP $\mathcal{P}$ into an equivalent (*) CSP. It is

COMPLETE if, given an input CSP $\mathcal{P}$, it always transforms $\mathcal{P}$ in a finite disjunction of simple (solved form) CSP, globally equivalent to $\mathcal{P}$ (or to `false` if it is unsatisfiable). A solved form constraint is an implicit description of its solutions.

INCOMPLETE $\mathcal{P}$ is transformed in a simpler equivalent CSP (*) but solutions are implicit (i.e., they can not determined by directly combining domain values).

(*) Equivalent CSPs may contain a larger number of variables. Admissible solutions sets, restricted to the same variables, are equal.

A constraint solver is based on the repeated application of proof rules

$$\frac{\varphi}{\psi}$$

where $\varphi$ and $\psi$ are CSPs.
$\psi$ can contain new variables not occurring in $\varphi$. A rule is said *equivalence preserving* if $Sol(\varphi) = Sol(\psi)|_{\text{vars}(\varphi)}$,

The size of the domains is reduced by rules that exploiting the information stored in the constraints.
The rules have the form:

$$\frac{\varphi = \langle \mathcal{C}; \mathcal{D}_\in \rangle}{\psi = \langle \mathcal{C}'; \mathcal{D}'_\in \rangle}$$

- $\mathcal{D}_\in = X_1 \in \mathcal{D}_1, \ldots, X_k \in \mathcal{D}_k$,
- $\mathcal{D}'_\in = X_1 \in \mathcal{D}'_1, \ldots, X_k \in \mathcal{D}'_k$,
- for all $i = 1, \ldots, k$ it holds that $\mathcal{D}'_i \subseteq \mathcal{D}_i$, and
- $\mathcal{C}'$ is the restriction of $\mathcal{C}$ to the new domains.

Some constraints in $\mathcal{C}'$ can become redundant and hence removed (e.g. $\underline{X + 0 > 0}, X > 2$).
If $\mathcal{C}' = \emptyset$ then $\psi = \texttt{true}$ ($\mathcal{D}'_\in$ represents all the solutions).
If, instead, for some $i$ $\mathcal{D}'_i = \emptyset$ then $\psi = \texttt{fail}$.

$$\frac{\langle X < Y; X\ in\ 5..15, Y\ in\ 0..10\rangle}{\langle X < Y; X\ in\ 5..9, Y\ in\ 6..10\rangle}$$

Therefore:

$$\begin{aligned}\mathcal{C} = \mathcal{C}' = \ \{ \ & (5,6),(5,7),(5,8),(5,9),(5,10),\\ & (6,7),(6,8),(6,9),(6,10),\\ & (7,8),(7,9),(7,10),\\ & (8,9),(8,10),\\ & (9,10)\}\end{aligned}$$

Constraints are transformed/simplified. This process might introduce new variables (with a domain). `fail` can be introduced.

$$\frac{\varphi = \langle \mathcal{C}; \mathcal{D}_{\in} \rangle}{\psi = \langle \mathcal{C}'; \mathcal{D}'_{\in} \rangle}$$

- $\mathcal{D}_{\in} = X_1 \in \mathcal{D}_1, \ldots, X_k \in \mathcal{D}_k$,
- $\mathcal{D}'_{\in} = X_1 \in \mathcal{D}_1, \ldots, X_k \in \mathcal{D}_k$, plus the new (non-empty) domains for the new variables (existing domains are not reduced by these rules)

$$\frac{\langle e_1 \neq e_2; \mathcal{D}_\in \rangle}{\langle X = e_1, X \neq e_2; \mathcal{D}_\in, X \; in \; \mathbb{Z} \rangle}$$

The new variable $X$ is introduced, possibly with a large domain.
New variables introduction is the technical reason why we refer to equivalence preserving (*) of CSP (variables, domains and constraints can vary)

Sometimes we need (or prefer) to introduce new constraints (typically weaker than existing ones):

$$\frac{\varphi = \langle \mathcal{C}; \mathcal{D}_{\in} \rangle}{\psi = \langle \mathcal{C} \cup \{C\}; \mathcal{D}_{\in} \rangle}$$

where $C$ is a new constraint

Consider this rule with variables on $\mathbb{R}$

$$\frac{\langle X^2 - 2XY + Y^2 < 5; X > 0, Y > 0\rangle}{\langle X^2 - 2XY + Y^2 < 5, X - Y < 3; X > 0, Y > 0\rangle}$$

The new constraints does not introduce new information being implied logically by the others. However, simple constraints allow a more effective constraint based reasoning.

- The repeated application of a set of rules (domain reduction, transformation, introduction) is said constraint propagation
- A *derivation* is a sequence of application of the proof rules (with a suitable variable renaming when instantiating the rules to the actual CSP)
- A finite derivation is
  - *failing* if the last CSP is `fail`,
  - *stable* if it is not failing and no rule can be applied to the reached CSP
  - *successful* if it is stable and the final constraint is in solved form

As general consideration, constraint propagation MUST be computationally efficient (polynomial time) at every rule application

# Local Consistency

## NODE CONSISTENCY

A CSP $\mathcal{P}$ is *node consistent* if for each variable $X$ in $\mathcal{P}$, every unary constraint on $X$ coincides with the domain of $X$.

$$\langle X_1 \geq 0, X_2 \geq 0; X_1 \ in \ \mathbb{N}, X_2 \ in \ \mathbb{N} \rangle$$

is node consistent

$$\langle X_1 \geq 5, X_2 \geq 1; X_1 \ in \ \mathbb{N}, X_2 \ in \ \mathbb{N} \rangle$$

is not (but it is consistent—i.e. it admits solutions)

$$\langle X_1 \neq X_2, X_1 = 0, X_2 = 0; X_1 \ in \ 0..0, X_2 \ in \ 0..0 \rangle$$

is node consistent, but not consistent.

## NODE CONSISTENCY

A CSP $\mathcal{P}$ is *node consistent* if for each variable $X$ in $\mathcal{P}$, every unary constraint on $X$ coincides with the domain of $X$.

$$\langle X_1 \geq 0, X_2 \geq 0; X_1 \; in \; \mathbb{N}, X_2 \; in \; \mathbb{N} \rangle$$

is node consistent

$$\langle X_1 \geq 5, X_2 \geq 1; X_1 \; in \; \mathbb{N}, X_2 \; in \; \mathbb{N} \rangle$$

is not (but it is consistent—i.e. it admits solutions)

$$\langle X_1 \neq X_2, X_1 = 0, X_2 = 0; X_1 \; in \; 0..0, X_2 \; in \; 0..0 \rangle$$

is node consistent, but not consistent.

## NODE CONSISTENCY

A CSP $\mathcal{P}$ is *node consistent* if for each variable $X$ in $\mathcal{P}$, every unary constraint on $X$ coincides with the domain of $X$.

$$\langle X_1 \geq 0, X_2 \geq 0; X_1 \ in \ \mathbb{N}, X_2 \ in \ \mathbb{N} \rangle$$

is node consistent

$$\langle X_1 \geq 5, X_2 \geq 1; X_1 \ in \ \mathbb{N}, X_2 \ in \ \mathbb{N} \rangle$$

is not (but it is consistent—i.e. it admits solutions)

$$\langle X_1 \neq X_2, X_1 = 0, X_2 = 0; X_1 \ in \ 0..0, X_2 \ in \ 0..0 \rangle$$

is node consistent, but not consistent.

## NODE CONSISTENCY

A CSP $\mathcal{P}$ is *node consistent* if for each variable $X$ in $\mathcal{P}$, every unary constraint on $X$ coincides with the domain of $X$.

$$\langle X_1 \geq 0, X_2 \geq 0; X_1 \ in \ \mathbb{N}, X_2 \ in \ \mathbb{N} \rangle$$

is node consistent

$$\langle X_1 \geq 5, X_2 \geq 1; X_1 \ in \ \mathbb{N}, X_2 \ in \ \mathbb{N} \rangle$$

is not (but it is consistent—i.e. it admits solutions)

$$\langle X_1 \neq X_2, X_1 = 0, X_2 = 0; X_1 \ in \ 0..0, X_2 \ in \ 0..0 \rangle$$

is node consistent, but not consistent.

Node consistency can be achieved by repeating the application of the following *domain reduction* rule:

$$\frac{\langle C; X \ in \ D_X \rangle}{\langle C; X \ in \ D_X \cap C \rangle}$$

where $C$ is a unary constraint on $X$.

Complexity $O(cd)$ where $c = |C|, d = \max_i |D_i|$.

Node consistency can be achieved by repeating the application of the following *domain reduction* rule:

$$\frac{\langle C; X \ in \ D_X \rangle}{\langle C; X \ in \ D_X \cap C \rangle}$$

where $C$ is a unary constraint on $X$.

Complexity $O(cd)$ where $c = |C|, d = \max_i |D_i|$.

Node consistency can be achieved by repeating the application of the following *domain reduction* rule:

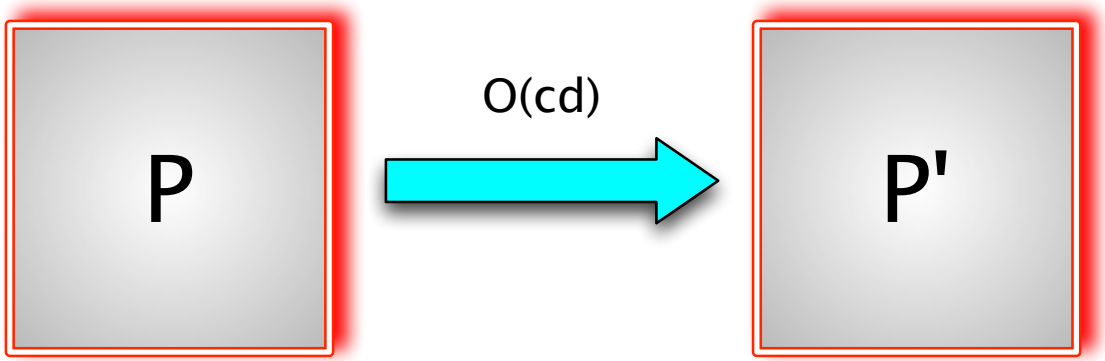
$$\frac{\langle C; X \ in \ D_X \rangle}{\langle C; X \ in \ D_X \cap C \rangle}$$

where $C$ is a unary constraint on $X$.

Complexity $O(cd)$ where $c = |C|, d = \max_i |D_i|$.

# Constraint Propagation

- We have seen the node consistency property and how to achieve it.
- The domain reduction rule for Node Consistency never increase domains (it is, in a sense, monotone). This property ensures that the order of rule application is immaterial.




- Let us see now the arc consistency property.

## ARC CONSISTENCY

A binary constraint $C$ on the variables $X$ and $Y$ with domains $D_X$ and $D_Y$ is *arc consistent* if:

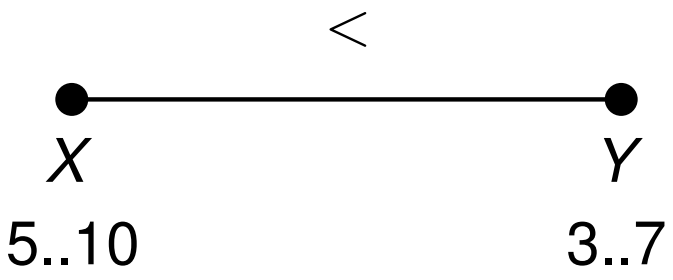
1. $(\forall a \in D_X)(\exists b \in D_Y)\,((a, b) \in C)$, and
2. $(\forall b \in D_Y)(\exists a \in D_X)\,((a, b) \in C)$.

A CSP $\langle \mathcal{C}; \mathcal{D}_{\in} \rangle$ is arc consistent if all binary constraints in $\mathcal{C}$ are arc consistent.

The words Arc and Node comes from graphs, of course. It is common to deal with a constraint graph where nodes are variables and arcs are binary constraints between the two variables.

$$\langle X < Y; X\ in\ 5..10, Y\ in\ 3..7 \rangle$$

can be represented by the graph:




is not arc consistent (but it is consistent)

$$\langle X \neq Y, Y \neq Z, X \neq Z; X\ in\ 0..1, Y\ in\ 0..1, Z\ in\ 0..1 \rangle$$

is arc consistent, but not consistent.

Arc consistency can be obtained by repeating the application of the following *domain reduction* rules:

$$(AC1) \qquad \frac{\langle C; X \ in \ D_X, Y \ in \ D_Y \rangle}{\langle C; X \ in \ D'_X, Y \ in \ D_Y \rangle}$$

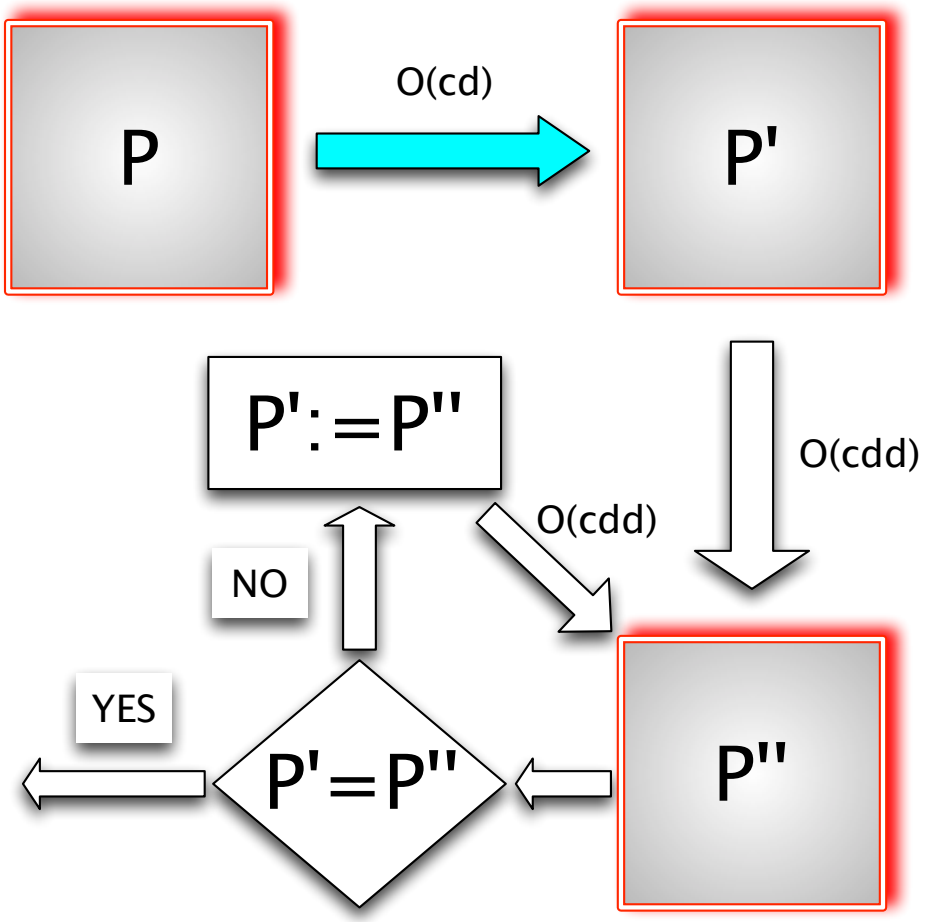
$$(AC2) \qquad \frac{\langle C; X \ in \ D_X, Y \ in \ D_Y \rangle}{\langle C; X \ in \ D_X, Y \ in \ D'_Y \rangle}$$

where $C$ is a binary constraints on $X$ and $Y$:

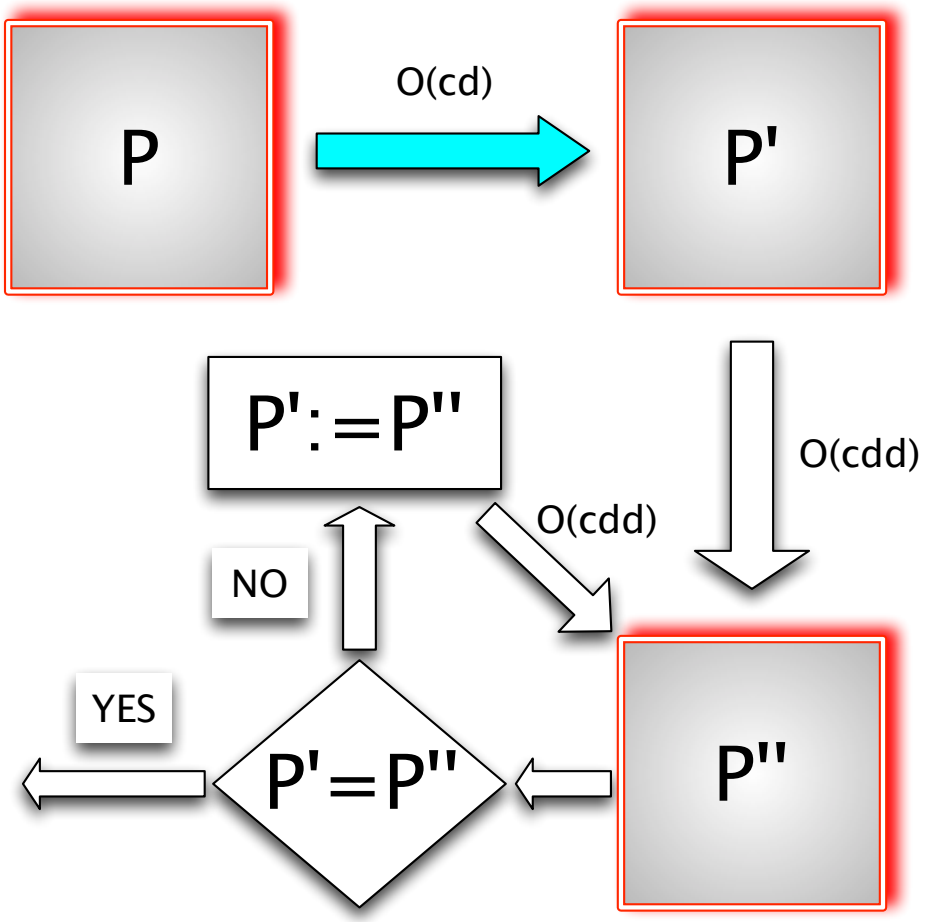
- $D'_X = \{a \in D_X \ : \ (\exists b \in D_Y)((a, b) \in C)\}$
- $D'_Y = \{b \in D_Y \ : \ (\exists a \in D_X)((a, b) \in C)\}$

# Constraint Propagation

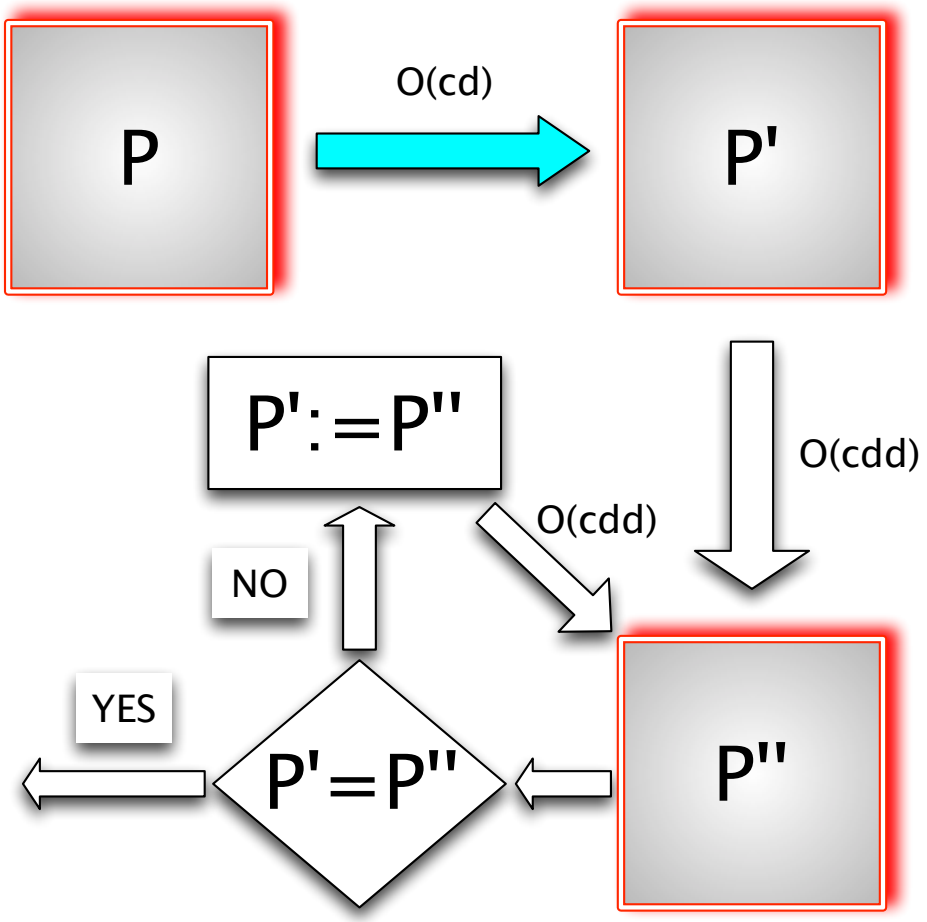
## Arc Consistency—fixpoint procedure (AC1)




Complexity? $O(ncd^3)$ —$n$ is the number of variables



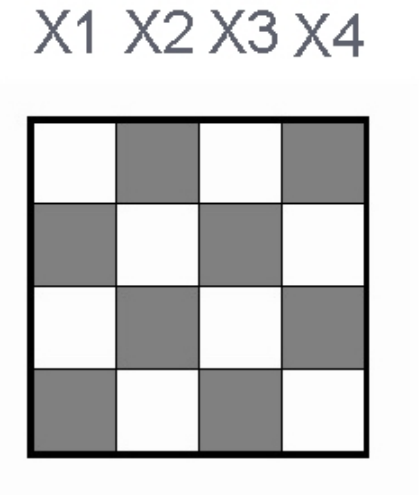

Complexity? $O(ncd^3)$ —$n$ is the number of variables




Complexity? $O(ncd^3)$ —$n$ is the number of variables

# Reasoning with Constraint Propagation

## Arc Consistency, $X_1 = 1$



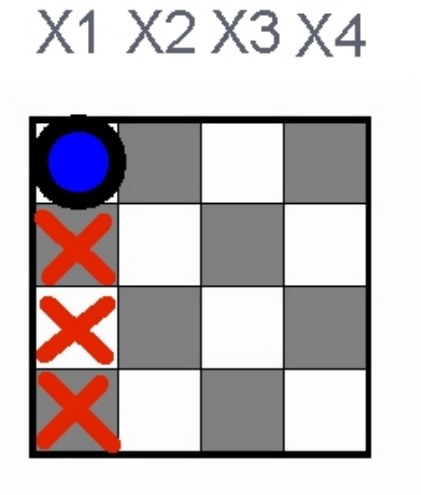

$$X_1, \ldots, X_4$$

$$D(X_i) = \{1, \ldots, 4\} \text{ for } i = 1..4$$

$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$

# Reasoning with Constraint Propagation

## Arc Consistency, $X_1 = 1$



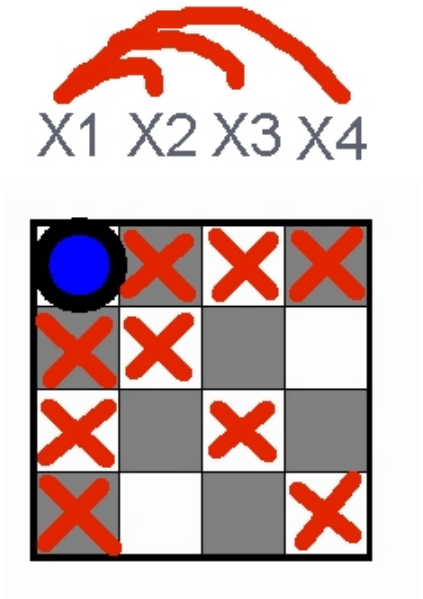

$$X_1, \ldots, X_4$$

$$D(X_1) = \{1\}, D(X_i) = \{1, \ldots, 4\} \text{ for } i = 2..4$$

$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$

# Reasoning with Constraint Propagation

## Arc Consistency, $X_1 = 1$



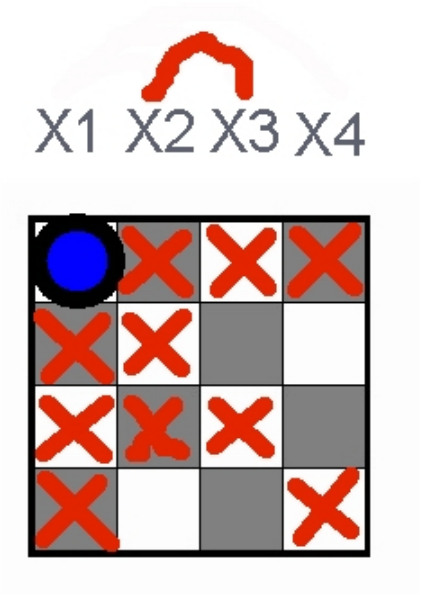

$$X_1, \ldots, X_4$$

$$D(X_1) = \{1\}, D(X_2) = \{3, 4\}, D(X_3) = \{2, 4\}, D(X_4) = \{2, 3\}$$

$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$

# Reasoning with Constraint Propagation

## Arc Consistency, $X_1 = 1$



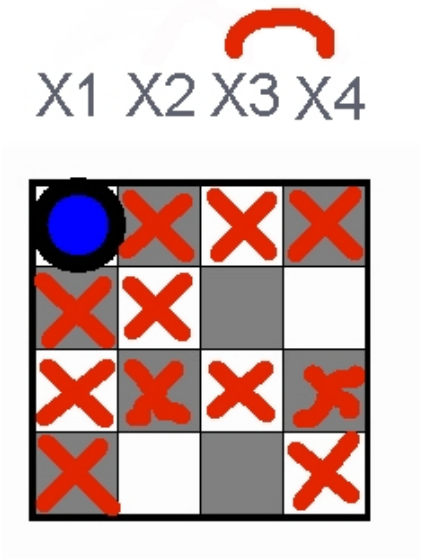

$$X_1, \ldots, X_4$$

$$D(X_1) = \{1\}, D(X_2) = \{4\}, D(X_3) = \{2, 4\}, D(X_4) = \{2, 3\}$$

$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$

# Reasoning with Constraint Propagation

## Arc Consistency, $X_1 = 1$



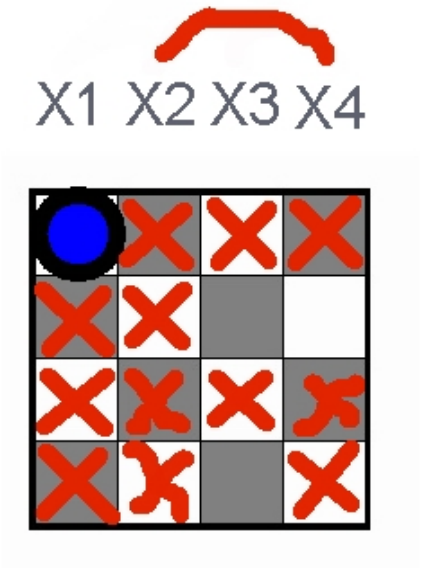

$$X_1, \ldots, X_4$$

$$D(X_1) = \{1\}, D(X_2) = \{4\}, D(X_3) = \{2, 4\}, D(X_4) = \{2\}$$

$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$

# Reasoning with Constraint Propagation

## Arc Consistency, $X_1 = 1$



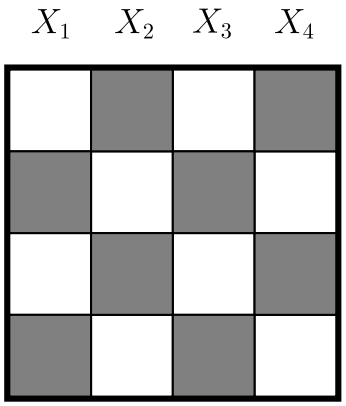

$$X_1, \ldots, X_4$$

$$D(X_1) = \{1\}, D(X_2) = \emptyset, D(X_3) = \{2, 4\}, D(X_4) = \{2, 3\}$$

$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$

# Reasoning with Constraint Propagation

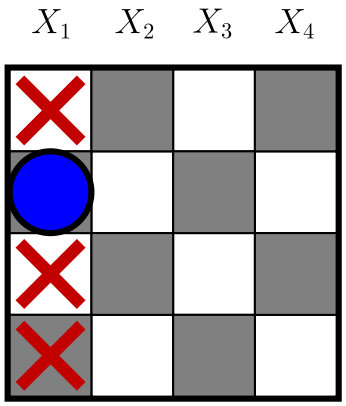
## Arc Consistency: $X_1 = 2$




$$X_1, \ldots, X_4$$

$$D(X_i) = \{1, \ldots, 4\} \text{ for } i = 1..4$$

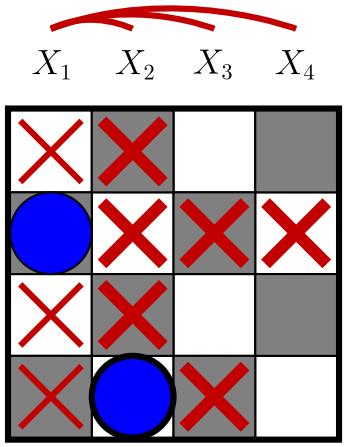
$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$




$$X_1, \ldots, X_4$$

$$D(X_1) = \{2\}, D(X_i) = \{1, \ldots, 4\} \text{ for } i = 2..4$$

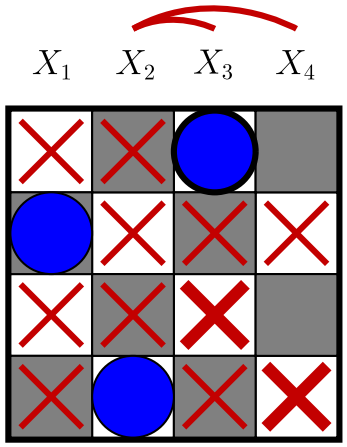
$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$




$$X_1, \ldots, X_4$$

$$D(X_1) = \{2\}, D(X_2) = \{4\}, D(X_3) = \{1, 3\}, D(X_4) = \{1, 3, 4\}$$

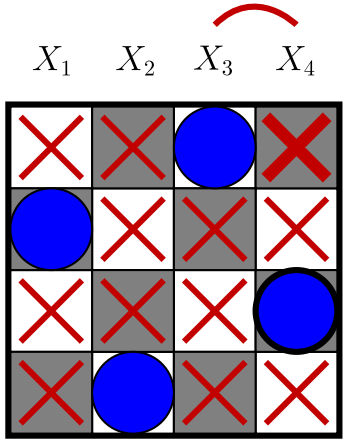
$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$




$$X_1, \ldots, X_4$$

$$D(X_1) = \{2\}, D(X_2) = \{4\}, D(X_3) = \{1\}, D(X_4) = \{3, 4\}$$

$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$




$$X_1, \ldots, X_4$$

$$D(X_1) = \{2\}, D(X_2) = \{4\}, D(X_3) = \{1\}, D(X_4) = \{3\}$$

$$X_i \neq X_j, |X_i - X_j| \neq (j - i) \text{ for } i < j$$

# Summary (till now)

- $n$ variables, $c$ constraints, $d$ maximum domain.
- Node Consistency: $O(cd)$
- Arc Consistency: $O(ncd^3)$
- Arc consistency rules only reduce the size of the domains: order of application are not important. At the end a (unique) fixpoint is reached.
- Exercise: optimize the fixpoint procedure. Have a look in the literature to AC3, AC4, AC5, ...